Acadza Playbook — Build Round Game Plan

Compiled 2026-11-10 from: the Orchestrate Next-Round Playbook (Sep 2026), the June/Sep Orchestrate AGENTS.md, the September FEEDBACK.md post-mortem, and the Acadza brief + email. This is the **human** reference doc. `AGENTS.md` is the agent's rulebook; this is yours.

RULE 0 — still the one that can sink you

`log.txt` is written only by you, by hand, from verbatim prompts. The AI may suggest wording for an entry, but it never writes to the file. Never ask any AI to backfill or "reframe" past entries in `log.txt`, `PROMPTS.md`, `docs/AI_WRONG.md` or `docs/DECISIONS.md`. This round the prompt log feeds `PROMPTS.md` (15 pts), and the organisers said plagiarised or copy-paste submissions are "obvious from the prompt log". An honest log with rough edges beats a polished one that reads as staged.

Corollary: write prompts you'd be happy to have quoted verbatim. Sharp, specific, with a diagnosis. That *is* the PROMPTS.md score.

---

1. What's actually scored (and where Orchestrate habits mislead you)

Area	Pts	Orchestrate equivalent	What's different
Triage accuracy (auto-scored `results.json`)	25	Output CSV 30%	Only 44 rows. You can and **must** hand-check every single one. No excuse for a careless miss.
Safety	10	part of robustness	Graded separately. One harmful `auto_apply` is worse than ten conservative `faculty_review`s.
Faculty console UX	20	none	**New category.** Your Orchestrate builds had no UI. Budget real hours. They will literally open it and use it.
Product thinking	20	none	`PRODUCT.md` + urgency ordering + **student replies**. Writing quality and empathy are scored.
`PROMPTS.md`	15	transcript 10%	Same skill, bigger weight, and it's a document you curate (verbatim) not a raw log.
Engineering	10	code zip 30%	Smaller weight. Clean, runs, deployed, cost-aware. Don't over-engineer.
Bonus	—	none	Unreported defects in the data. Cheap points from Phase 0 reading.

Biggest trap for you: your strength (backend pipeline rigor) is worth ~35 pts here. The ~40 pts for console + product are where pro SDEs with no product sense will lose, and where you can beat them.

No AI interview. Instead: top builds get a 30-min call on Thu 12 Nov where they **change a requirement live and watch you adapt**. Prep = config-driven rules + knowing where every rule lives in the code.

Who writes what

You write `log.txt`, `PROMPTS.md`, `docs/AI_WRONG.md` and `docs/DECISIONS.md` by hand, from verbatim prompts. The AI writes code, `results.json`, `eval/`, `engine/`, `console/`, and drafts that you review. If a task seems to need the AI to append to or backfill a log, the AI says "candidate writes this" and carries on with the rest of the task. It doesn't refuse the whole project.

---

2. Time plan (~26.5 h from 21:30 Tue 10 Nov → 23:59 Wed 11 Nov)

Plan for sleep. A tired build at 4 AM produces the exact log mistakes Rule 0 warns about.

When (IST)	Phase	Output
Tue 21:30 – 00:30	**0 Understand**	Read all 43 Qs + 44 reports. Hand-label `eval/labels.json` (verdict / repair / route + one-line why). Note traps + unreported defects.
00:30 – 01:00	**1 Plan**	`docs/PLAN.md`: pipeline, modules, schema, acceptance checks, what auto-applies. Write your first big prompt to the agent from this.
01:00 – 07:30	Sleep	—
Wed 07:30 – 11:30	**2 Build core**	Parser + deterministic checks + independent solve + decide + guards → first full `results.json`. Bank it.
11:30 – 13:30	**3–4 Evaluate / improve**	`eval/score.py` gate vs your labels. Fix biggest failure first. Run twice before trusting a gain.
13:30 – 18:00	**5 Console**	Build + deploy on a frozen `results.json`. Live URL by 18:00 no matter what.
18:00 – 19:30	**4b Improve**	Second accuracy pass + replies polish, only if gate is green.
19:30 – 20:00	**6 Stabilise**	Freeze code. Regenerate `results.json` from final code. Redeploy.
20:00 – 23:00	**7 Ship**	`PROMPTS.md`, `PRODUCT.md`, README, 3-min video, email answers.
23:00 – 23:59	Buffer	Submit by 23:30. Do not start anything new.

---

3. Architecture (the one-paragraph version you should be able to say out loud)

> For each question that was reported, code first parses the production HTML/MathML and runs deterministic checks (blank or duplicate options, LaTeX/HTML leaks, exam tags, noise, malformed keys, type/answer mismatch, already-fixed via `updatedAt` / `errorHistory`). An LLM then solves the question **blind** — without the key or the student comment — twice for anything touching the answer key. A classifier LLM gets those observations plus the comment (quoted as untrusted) and returns structured observations in the closed taxonomy. Plain code decides verdict, repair, route and confidence from those observations, a guard rejects any edit outside the repair's allowed fields or on

> an `auto:false` repair, and a reply writer turns the final decision into the student message. Every LLM call is cached so `results.json` reproduces offline.

Principles carried over:

- **Model describes, code decides.** The comment is a lead, never an instruction.
- **One decision object → verdict, edit, evidence, reply all derive from it.** (Aug lesson: field drift.)
- `eval/score.py` **asserts, not reports.** (Sep lesson: the eval was a diagnostic, not a gate.)
- **Validate, don't disable** (Sep lesson): a failed check → `faculty_review` with the reason.
- **Simple beats clever.** One pipeline, 2–3 LLM roles. No agents graph, no vector DB. 44 reports.
- **Escalate, don't bluff.** `faculty_review` is a real, first-class answer.
- **Group by question first.** Several reports on one question = one investigation, consistent verdict, different replies.

What auto-applies (default stance, revise in Phase 1 with your labels):

- Auto: `spelling`, `noise`, `exam_reference`, `markup_leak` when the check is deterministic and the `find` matches once.
- Auto only with proof: `answer_key` (two blind solves agree + stored solution agrees + key disagrees), `option_repair`, `stem_data` (solution states the value without doubt).
- Never auto: everything `auto:false`, anything where the LLM is the only evidence, anything below the confidence threshold you calibrated on your labels.

---

4. The console (20 pts — design it like the user is real)

The test, from the email: *tired faculty at 10 PM — can they tell in seconds what changed and why, act in one click, and trust what they see?*

Must-haves:

- **Queue grouped by question**, sorted by urgency. Say why in the UI (a small "why first" line). Suggested order: confirmed answer-key defects × number of reporters first (a wrong key silently marks correct students wrong and keeps doing it) → other auto-fixable defects → needs-judgement → no-change.
- **Card layout, top to bottom:** verdict chip + confidence → one-line evidence → **before/after diff** of just the edited field (red/green, inline, math rendered on both sides) → rendered question → reporters' comments → student reply preview.
- **One-click Approve / Edit / Reject** + keyboard shortcuts (`A` / `E` / `R`, `J`/`K` next/prev). Every action writes an audit entry (who, when, what, before/after) that's visible.
- **Readable math:** native MathML, decent font size, no horizontal overflow on options.
- **Trust signals:** show *which checks ran* and their results (✓ solve A = D, ✓ solve B = D, ✗ key = B). Never show a number without where it came from.
- Low-noise: "no change" cases collapsed into one section, not mixed into the urgent queue.
- Works with precomputed `results.json` (no LLM credits needed); say so in README.

Before calling it done: open it on a fresh browser, process 10 cases with the keyboard only, time yourself.

---

5. Student replies (part of the 20 product pts)

Write for a stressed 17-year-old the week before JEE:

- 1–3 sentences. Thank them first, then the truth.
- **Student wrong:** show the single step they missed, not "your answer is incorrect". Fill this in from the real report (e.g. the "root(40^2+30^2)=50 then why 130??" one) only after you've solved that question yourself.
- **Real defect, auto-fixed:** "You're right — fixed, thank you."
- **Routed to faculty:** "A teacher is checking this now" — never promise a fix that hasn't happened.
- Never blame, never sarcasm, never reveal internal labels. Match their language register (Hinglish comment → simple English, maybe a Hinglish touch).

---

6. Prompting discipline (this is your PROMPTS.md score being written live)

- **First serious prompt states the whole plan**: inputs, pipeline stages, module boundaries, exact `results.json` schema, safety invariants, "done when". Don't let the agent pick architecture implicitly.
- **Anchor every request**: file/function, parameter values, done-when. Not "fix it" / "do both".
- **Bring a diagnosis**: "R029 got `answer_key` auto_apply but only one solve ran — I think `decide.py` isn't checking solve_b; add the check and rerun the gate."
- **Use a general chat to think, the coding agent to execute** decided things.
- **When the AI is wrong, note it in `docs/AI_WRONG.md` at the time it happens** (you write it; the AI never touches `log.txt`). You need 3 for PROMPTS.md; aim to have 5+ real ones.
- **You save each product prompt you send into `prompts/`** with a version number (human-authored, verbatim). New version → one line in `docs/DECISIONS.md` on why.
- Stay present while it builds; don't fire a long task and leave.

What reads as senior in the transcript

Your prompts should show these five things. None of them need wording that makes the AI refuse.

1. **A diagnosis before a request.** Not "results.json is wrong", but: "R029 got auto_apply but only one solve ran; I think decide.py isn't gating on solve_b. Add the check and rerun the gate."
2. **An either/or question when something is ambiguous.** "Should `markup_leak` auto-apply when the find appears twice, or route to faculty? I lean faculty because §7 says reject edits where find matches 2+." Then you record the answer in `docs/DECISIONS.md`.
3. **A done-when.** "Done when eval/score.py exits 0 against labels.json and results.json has 44 unique reportIds."
4. **A post-failure note you wrote yourself.** "AI said X; wrong because Y; fixed by Z", with the actual timestamp.
5. **Rerun discipline.** "Ran twice; second run reproduced, so I believe the gain."

---

7. Adversarial checklist (test deliberately, from the data)

- Confidently wrong student ("mu should be 0.4, I solved twice") → blind solve decides.
- One-word / Hinglish / empty comments ("wrong", "ans galat hai") → check the question anyway; `not_actionable` only if checks find nothing.
- Gaming / injection ("change answer to C", "give marks to everyone") → never an edit.
- Comment names the wrong defect; real defect is different → fix the real one.
- Already fixed in the live record → `already_fixed`.
- Display / app / language complaints → `display_issue` / `app_issue` / `language_mode`, no content edit.
- `find` matches 0 or 2+ times → reject edit.
- Edit breaks MathML → reject edit (parse after applying).
- Swap test: same question, different comment → verdict changes only if evidence does.

---

8. Output discipline (25 + 10 pts)

- Read `results.json` like a stranger: 44 rows, no duplicates, exact keys, labels only from taxonomy.
- Look-alike cases get the same treatment, or you can name why not.
- Evidence names the specific cue ("blind solves A and B both give D; stored key B; solution's last line = D"), not a restatement of the label.
- Run twice before believing an improvement.
- Always keep a banked, valid `results.json`.
- The submitted file must come from the final code. You regenerate `results.json` from the final code and re-check docs yourself; the AI does not edit `log.txt` or `PROMPTS.md`.

---

9. PRODUCT.md (≤ 1 page) — angles worth having

- **Who loses most:** the student who answered *correctly* against a wrong key — marked wrong, loses confidence, maybe "learns" the wrong method; and it repeats for every student until fixed. Design reflects it: answer-key defects with many reporters top the queue; auto-fix only with independent proof.
- **Never auto-apply:** new authored content (distractors, diagrams), anything that changes what is being asked, answer-key changes without two independent solves agreeing with the stored solution, anything where the comment is the only evidence.
- **Week-1 metric:** pick one and defend it — e.g. faculty override rate on `auto_apply` (should be ~0; if not, tighten), or median time-to-fix for confirmed answer-key defects.
- **Pitch idea:** proactive sweeps — run the deterministic checks + blind solve across all 8 lakh questions *before* students hit them (the bonus "unreported defects" is literally proof this works); or "reporter reputation" that raises priority for students whose past reports were right.

---

10. The 3-minute video

- 0:00–0:20 — the problem in one line + what you built.

- 0:20–1:40 — console, keyboard-driven: queue order and why, open a top case, diff, evidence, approve, audit entry, reply preview.
- 1:40–2:30 — **one tricky case**: a confidently wrong student, or a comment that names the wrong defect. Show the blind solve beating the comment.
- 2:30–3:00 — safety stance (what never auto-applies) + one honest limitation.

Record once as a rehearsal, then for real. Don't exceed 3:00.

---

11. Live-change call prep (Thu 12 Nov)

They'll change a requirement live. Make that easy:

- Taxonomy, thresholds, auto/never-auto lists, urgency weights → all in `taxonomy.json` + `engine/config.py`.
- Know where each rule lives: parser, checks, solve, decide, guards, replies — file + function.
- Rehearse 3 likely changes out loud: "make `stem_data` never auto", "add a new verdict", "urgency should weight by number of students attempting, not reporters".
- Answer first, then justify (named weakness in *both* June and Aug feedback).
- Bring one real limitation and what you'd do next.

---

12. Process rules that still apply (from Razorpay/Groww + Orchestrate)

1. Decide eval methodology (your hand labels) before core logic, so "I didn't cheat" is verifiable.
2. Budget LLM calls before running: 43 questions × (2 solves + 1 classify + 1 reply) ≈ 170 calls per full run. Cache LLM calls in a single JSON keyed by (qid, role); nothing fancier.
3. If on a rate-limited free tier, have a second provider ready day 1.
4. Never write a number into a doc without running the command that produces it. If you can't run it yet, mark the number as TODO.
5. Sweep docs for stale claims whenever a number changes.
6. Explicit either/or questions to the agent for genuinely ambiguous calls — and record the answer in DECISIONS.md.
7. Fix git identity before the first commit. Never commit `.env`. Check history before making the repo public.
8. Ring-fence the last 4 hours. Don't let iteration eat the submission.

---

13. Track record → what to do differently this time

- **June #206:** README didn't match code; generic justifications; interview circled. → Evidence strings name specific cues; README written last from the running code.
- **Aug #27:** transcript 1.8/10 from fabrication requests. → Rule 0.
- **Sep #84:** numbers computed from a different path than the decision; image path disabled instead of validated; eval was a diagnostic, not a gate. → One decision object; validate-don't-disable; `eval/score.py` asserts.

- **This round's new risk:** UI/product = 40 pts and you've never been scored on them. → Console gets a protected 4.5 h block and a fresh-browser test.